

UPPRESSO: Untraceable and Unlinkable Privacy-PREserving Single Sign-On Services

Chengqian Guo, Jingqiang Lin, Quanwei Cai, Wentian Zhu, Wei Wang, Jiwu Jing, Qiongxiao Wang, Bin Zhao, and Fengjun Li

Abstract—Single sign-on (SSO) protocols such as OpenID Connect (OIDC), allow a user to maintain only the credential for an identity provider (IdP) to log into multiple relying parties (RPs). However, SSO introduces privacy threats, as (a) a curious IdP could trace a user’s visits to RPs, and (b) colluding RPs could learn a user’s online profile by linking her identities across these RPs. This paper presents a privacy-preserving SSO scheme, called UPPRESSO, to protect a user’s online profile against both (a) an honest-but-curious IdP and (b) malicious RPs colluding with some users. An identity-transformation approach is proposed to generate untraceable ephemeral pseudo-identities for an RP and a user in each login, from which the visited RP derives a permanent account for the user, while these identity transformations provide unlinkability among the logins visiting different RPs. We built the UPPRESSO prototype providing OIDC-compatible SSO services, accessed from a commercial-off-the-shelf browser without installing any extension. The extensive evaluations show that it fulfills the security and privacy requirements of SSO with reasonable overheads.

Index Terms—Single sign-on, security, privacy.

I. INTRODUCTION

Single sign-on (SSO) protocols such as OpenID Connect (OIDC) [1], OAuth 2.0 [2], and SAML [3], [4], are widely deployed for identity management and authentication. SSO allows a user to log into a website from browsers, known as the *relying party* (RP), using her account registered at another trusted web service, known as the *identity provider* (IdP). An RP delegates its user identification and authentication to the IdP, which issues an *identity token* (such as “id token” in OIDC and “identity assertion” in SAML) for a user to visit the RP. For instance, when an OIDC user initiates a login to visit an RP, the target RP constructs an identity-token request with its identity and redirects it to a trusted IdP. After authenticating this user, the IdP issues an identity token binding the identities of the authenticated user and the target RP (i.e., ID_U and ID_{RP}), which is returned to the user and then forwarded to the RP. Finally, the RP verifies the identity token to determine whether the token holder is allowed to log in. Thus, a user maintains only one credential for user authentication to the IdP, instead of multiple credentials for visiting different RPs.

Chengqian Guo is affiliated with Yuncheng Vocational and Technical University, China. Jingqiang Lin (corresponding author, linjq@ustc.edu.cn), Wentian Zhu, and Wei Wang are affiliated with University of Science and Technology of China (USTC), China. Quanwei Cai is with Beijing Zitiao Network Technology Co., Ltd, China. Jiwu Jing is with University of Chinese Academy of Sciences (UCAS), China. Qiongxiao Wang is with Beijing Certification Authority (BJCA) Co., Ltd, China. Bin Zhao is with Tencent America, USA. Fengjun Li is with the University of Kansas, USA.

Manuscript received September 23, 2025.

The wide adoption of SSO raises concerns on user privacy, because it facilitates the tracking of a user’s login activities [5], [6], [7], [8]. To issue identity tokens, an IdP should know the target RP to be visited by a user. So a curious IdP could trace a user’s all login activities [7], [6], called *IdP-based login tracing* in this paper. Another privacy threat arises from the fact that RPs learn a user’s identity from the identity tokens they receive. If an *identical* user identity is enclosed in the tokens for a user to visit different RPs, colluding RPs could link the logins across these RPs to learn the user’s online profile [8], [9]. This is called *RP-based identity linkage*.

This paper presents UPPRESSO, an Untraceable and Unlinkable Privacy-PREserving Single Sign-On protocol. We propose *identity transformations* for SSO services, and integrate them into OIDC. When visiting an RP in UPPRESSO, a user transforms ID_{RP} into *ephemeral* PID_{RP} , which is sent to an IdP to transform ID_U into an *ephemeral* user pseudo-identity PID_U . Then, the identity token issued by the IdP binds only PID_U and PID_{RP} , instead of ID_U and ID_{RP} . On receiving the token, the target RP transforms PID_U into an account *unique at each RP but identical across multiple logins* visiting this RP.

UPPRESSO prevents both IdP-based login tracing and RP-based identity linkage, while existing privacy-preserving SSO schemes address only one of them [7], [6], [5], [10], [11] or introduce another fully-trusted component in addition to the IdP [12], [6], [13]. In Section II we analyze these solutions, and also compare UPPRESSO with privacy-preserving identity federation [14], [15], [16], [17], [18], [19] where browser extensions are required. UPPRESSO offers an easy-to-deploy option to enhance the protections of user privacy for SSO, because no extra component is introduced and the services are accessed from commercial-off-the-shelf (COTS) browsers without extensions. Our contributions are as follows.

- We proposed a novel identity-transformation approach for privacy-preserving SSO services and designed identity-transformation algorithms with desirable properties.
- We designed the UPPRESSO protocol by integrating the identity transformations into OIDC compatibly, and proved the security and privacy guarantees.
- We implemented the UPPRESSO prototype on top of an open-source OIDC implementation. Through performance evaluations, we confirmed that UPPRESSO introduces reasonable overheads.

We explain the privacy threats in SSO and present related works in Section II. The identity-transformation approach is

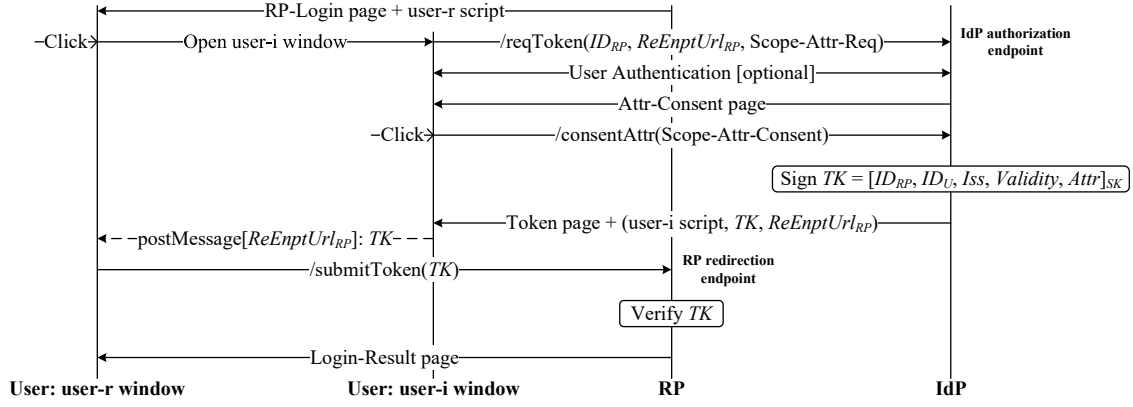


Fig. 1. The implicit SSO login flow of OIDC with pop-up UX

proposed in Section III, followed by the detailed designs in Section IV. Security and privacy are analyzed in Section V. We present the prototype implementation and evaluations in Section VI, and discuss extended issues in Section VII. Section VIII concludes this work.

II. BACKGROUND AND RELATED WORK

A. OpenID Connect

OIDC [1] is the most popular SSO protocol. It supports different login flows: implicit flow, authorization code flow, and hybrid flow (a mix of the other two). The flows differ in the steps for requesting, receiving and forwarding identity tokens, but have common security requirements for the tokens. We present our designs in the implicit flow and discuss the support for the authorization code flow in Section VII.

Users and RPs register at an IdP with their identities and other information such as user credentials (e.g., passwords) and RP *redirection endpoint URLs* to receive tokens denoted as $ReEnptUrl_{RP}$, respectively [2].

User operations in the login flows are conducted on a user agent (or browser typically). The flow with pop-up UX is proposed to provide an in-context user experience [20], [21], [22]. As shown in Figure I, when a user clicks the SSO login button in a browser window visiting the target RP (called the *user-r window*), an identity-token request is constructed with the RP's identity, the RP's redirection endpoint URL, and the scope of requested user attributes. It pops up another window (i.e., the *user-i window*), and this request is sent to the IdP's *authorization endpoint* where tokens are issued [2], [1]. After authenticating the user, the IdP issues an identity token that encloses the (pseudo-)identities of the authenticated user and the target RP, the user attributes, a validity period, etc. The token is then forwarded to the RP's redirection endpoint via the *user-i window*. Finally, the RP verifies the token and allows the token holder to log in as the enclosed user (pseudo-)identity. There are scripts in two browser windows (called the *user-r script* and the *user-i script*), responsible for communicating with the respective origin web servers and also the cross-origin communications by the `postMessage` HTML5 API [23] within the browser.

Alternatively, if a browser works as the user agent of OIDC but with only one window, user attributes are consented in

this window when redirected to the IdP [1], [2], [21]. This is called *redirect UX*. After a token is issued by the IdP, it is sent to the RP's redirection endpoint as a fragment (but not query parameters) of the URL due to security considerations, so a script from the RP is needed to handle this [24]. The UPPRESSO prototype is built with pop-up UX, and we discuss the support for redirect UX in Section VII.

B. Privacy-Preserving SSO and Identity Federation

In the original OIDC protocol an IdP signs identity tokens binding the identities of the authenticated user and the target RP [1], [2]. A user's login activities are leaked due to these identities: The curious IdP learns the visited RP from ID_{RP} enclosed in an identity-token request (i.e., IdP-based login tracing), while colluding RPs link a user's logins visiting these RPs by ID_U in the tokens (i.e., RP-based identity linkage).

Privacy-preserving SSO. Existing solutions propose various mechanisms to hide (a) RP identities from the IdP and (b) user identities from the colluding RPs. Table I compares these schemes, and only UPPRESSO provides services accessed from COTS browsers while preventing both of the two privacy threats and not introducing extra trusted components in addition to the honest IdP.

Pairwise pseudonymous identifiers (PPIDs) are specified to protect user privacy against colluding RPs [1], [4], [5]. An IdP assigns a unique PPID for a user to log into every RP and encloses it in identity tokens, so colluding RPs cannot link the user by PPIDs. It does not prevent IdP-based login tracing, because the IdP needs the RP's identity to assign PPIDs.

Other privacy-preserving schemes prevent IdP-based login tracing but leave users vulnerable to RP-based identity linkage, due to the identical user identities enclosed in identity tokens. For example, in BrowserID [7] after authenticating a user, an IdP issues a "user certificate" binding the user's identity to an ephemeral public key. The user then uses the corresponding private key to sign a subsidiary "identity assertion" that binds the target RP's identity and sends both of them to the RP. In SPRESSO an RP creates a one-time tag (i.e., ephemeral pseudo-identity) for each login [6], and in POIDC [11], [10] a user sends an identity-token request with a commitment on the target RP's identity (but not with plain ID_{RP}), which are enclosed in identity tokens along with ID_U .

TABLE I
PRIVACY-PRESERVING SOLUTIONS OF SSO AND IDENTITY FEDERATION

	Solution	Privacy Threat [‡]		Extra Trusted Component [†]	Accounts Derived from User Secret [‡]
		IdP-based Login Tracing	RP-based Identity Linkage		
SSO	OIDC w/ PPID [5]	○	●	●	●
	BrowserID [7]	●	○	●	●
	SPRESSO [6]	●	○	● ¹	●
	POIDC [11], [10]	●	○ ²	●	●
	MISO [12]	●	●	○	●
	UP-SSO [13]	●	●	●	○ ³
Identity Federation	PRIMA [25]	●	○	●	○
	PseudoID [14]	●	●	○	○
	Opaak [17]	●	●	●	○
	PP-IDF [15], [18], [16]	●	●	●	○
	Fabric Idemix [19]	●	●	●	○
SSO	UPPRESSO	●	●	●	●

[‡] Privacy Threat: ● prevented, ○ not prevented.

[†] Extra Trusted Component: ● no, ○ required.

[‡] Accounts Derived from User Secret: ● no, ○ yes.

1. SPRESSO assumes a malicious IdP, so a trusted forwarder server is introduced to decrypt the RP identity and forward tokens to the RP.
2. A variation of POIDC [11] proposes to also hide a user's identity in the commitment and prove this to the IdP in zero-knowledge, but it takes seconds or more to generate such a zero-knowledge proof (ZKP) even for a powerful server [26], [27], [28], which is impracticable for a user agent in SSO.
3. UP-SSO [13] runs a user-side Intel SGX enclave with a secret to calculate PPIDs.

MISO [12] decouples the assignment of PPIDs from an honest IdP, to an extra fully-trusted mixer server that calculates a user's PPID based on ID_U , ID_{RP} and a secret after it receives the user's identity from the IdP. MISO prevents RP-based identity linkage for the RPs receives only PPIDs, and IdP-based login tracing for ID_{RP} is disclosed to the mixer but not the IdP. It protects user privacy against even collusive attacks by the IdP and RPs, but the mixer could track a user's all login activities. Similarly, UP-SSO [13] runs a fully-trusted Intel SGX enclave on the user side, which is remotely attested by the IdP and receives the secret to generate PPIDs.

Privacy-preserving identity federation. Identity federation enables a user registered at a trusted IdP to be accepted by other parties, potentially with different accounts, but a long-term user secret is involved in the derivation of accounts,¹ so browser extensions are required to maintain this user secret.

In PRIMA [25], an IdP signs a credential that binds user attributes and a verification key. Using the corresponding signing key, the user authenticates herself and provides selected attributes to RPs. The verification key works as the user's global identity and exposes her to RP-based identity linkage. PseudoID [14] introduces another trusted server in addition to the trusted IdP, to blindly sign [32] a token that binds a long-term user secret and a pseudonym. The user then unblinds this token, presents it to the IdP which processes the secret, and then authenticates herself to an RP by this secret.

Privacy-preserving identity federation schemes are designed based on anonymous credentials [33], [34], [35]. In Opaak [17], UnlimitID [16], U-Prove [18], and EL PASSO [15], the IdP signs anonymous credentials, which bind a user secret, and

a user proves ownership of the anonymous credentials using her secret. Similarly, Fabric [19] integrates Idemix anonymous credentials [34] for unlinkable pseudonyms.

Some privacy-preserving solutions of identity federation [14], [15], [16], [17], [18], [19] prevent both IdP-based login tracing and RP-based identity linkage, for (a) the RP's identity is not enclosed in the anonymous credentials and (b) the user selects different pseudonyms to visit different RPs. They even protect user privacy against collusive attacks by the IdP and RPs, as these pseudonyms cannot be linked through anonymous credentials [33], [34], [35] when the ownership of these credentials is proved to colluding RPs.

Anonymous identity federation. Such approaches offer the strongest privacy, as a user visits RPs with pseudonyms that cannot be exploited to link any two actions. Anonymous identity federation was formalized and constructed using cryptographic primitives (e.g., ZKP and group signature) [31], [30], [29]. Extended features including proxy re-verification [31], designated verification [30] and distributed IdP servers [36], are also considered. These completely-anonymous authentication services only work for special scenarios and do not support user identification (or account uniqueness) at each RP, a common requirement in most applications.

C. OPRF-based Applications

PrivacyPass [37] and TrustToken [38] adopted the oblivious pseudo-random function (OPRF) of Hash(EC)DH [39], [40] to sign anonymous tokens. A user generates a random number e to blind an unsigned token T into $[e]T$. After authenticating the user, a token server signs $[e]T$ using its private key k and returns $[ke]T$. The user then uses e to convert it into $(T, [k]T)$, which is redeemed when she anonymously accesses resources. In this procedure, the token server obliviously calculates a pseudo-random output as an anonymous token, because $(T, [k]T)$ and $([e]T, [ke]T)$ cannot be linked.

UPPRESSO employs a similar cryptographic technique (see Section IV-C for details). A user selects t to transform ID_{RP} into $PID_{RP} = [t]ID_{RP}$. Then, the IdP uses the user's

¹ Although the term "single sign-on (SSO)" was used in some schemes [14], [17], [15], [29], [30], [31], this paper refers to them as *identity federation* to emphasize the difference. Such a user-maintained secret used to derive accounts in identity federation harm the user convenience: Once it is lost or leaked, the user has to notify all RPs to update her accounts derived from this secret and then the "singleness" property is broken, while SSO aims to minimize a user's interactions with RPs. Note that this user secret involved in the derivation of accounts is very different from the credential for user authentication to the IdP: If this credential is lost or leaked in SSO, a user renews it at the IdP only.

permanent identity u to calculate $PID_U = [u]PID_{RP} = [ut]ID_{RP}$, similar to token signing by the token server in PrivacyPass/TrustToken. IdP untraceability in UPPRESSO, i.e., the IdP cannot link ID_{RP} and $[t]ID_{RP}$, roughly corresponds to unlinkability of token signing-redemption, i.e., the token server cannot link T and $[e]T$ in PrivacyPass/TrustToken.

UPPRESSO leverages this cryptographic technique in very different ways.² First, it works among three parties, unlike the two-party PrivacyPass/TrustToken protocols. UPPRESSO shares the user-selected random number t with the target RP, enabling it to derive the user's permanent account, i.e., $Acct = [t^{-1}]PID_U$. This account is “obliviously” determined by the IdP when it calculates the user's pseudo-identity. Second, u and ID_{RP} , which roughly correspond to k and T in PrivacyPass/TrustToken, are assigned as the permanent identities of the user and the RP, respectively, to establish the relationship among identities and accounts. This is different from existing OPRF-based systems [37], [38], [42], [43], [44], [45], [46], [47], [48], [49] that always use k as a server's secret key. Finally, UPPRESSO leverages randomness of Hash(EC)DH OPRFs to ensure RP unlinkability, but this property is not utilized in PrivacyPass/TrustToken. This ensures colluding RPs cannot link any logins across RPs, even by sharing their knowledge about the pseudo-identities and permanent accounts in UPPRESSO. This property offers desirable indistinguishability of different users visiting colluding RPs. It roughly corresponds to the indistinguishability of different private keys for signing anonymous tokens, not considered in PrivacyPass/TrustToken.

Although UPPRESSO mathematically employs the same technique in the identity transformations as the Hash(EC)DH OPRF [39], [40], we require more properties of these algorithms. UPPRESSO essentially depends on the deterministic property of pseudo-random functions to correctly derive the account for any t , obliviousness to ensure IdP untraceability, and pseudo-randomness to ensure RP unlinkability. In contrast, PrivacyPass and TrustToken [37], [38] depend on only the properties of deterministicness and obliviousness. In fact, UPPRESSO requires additional properties for secure SSO services [50] (i.e., RP designation and user identification; see Theorems 2 and 1 in Section V-B), which are not discussed in OPRFs [51], [39], [40]. Thus, an OPRF is not always ready to work as the identity transformations in SSO.

III. THE IDENTITY-TRANSFORMATION APPROACH

In this section, we list the security requirements of SSO, and present the identity-transformation approach.

A. Security Requirements for SSO Services

Non-anonymous SSO services are designed to allow a user to log into an honest RP as her unique account at this RP, by presenting identity tokens issued by a trusted IdP [1], [2], [3], [4], [5]. This security goal is achieved through the following properties of identity tokens.

²We designed and implemented UPPRESSO in 2020 [41], and in 2023 someone reminded us that the identity transformations in UPPRESSO are very similar to the Hash(EC)DH OPRF.

TABLE II
THE (PSEUDO-)IDENTITIES IN THE IDENTITY TRANSFORMATIONS

Notation	Definition	Lifecycle
ID_{U_i}	The i -th user's unique identity at the IdP.	Permanent
ID_{RP_j}	The j -th RP's unique identity at the IdP.	Permanent
$Acct_{i,j}$	The i -th user's account at the j -th RP.	Permanent
$PID_{U_i,j}^l$	The i -th user's pseudo-identity in her l -th login visiting the j -th RP.	Ephemeral
$PID_{RP_j}^l$	The j -th RP's pseudo-identity in the user's l -th login visiting this RP.	Ephemeral

First of all, *authenticity*, *confidentiality*, and *integrity* of identity tokens are necessary to prevent forging, eavesdropping and tampering. In SSO services, identity tokens are signed by the trusted IdP, transmitted over HTTPS/TLS [1], [2], [3], and finally forwarded to the target RPs by the authenticated user. Common security mechanisms within a browser (e.g., the mechanisms of HTTP session and `postMessage` target-Origin) are also required for confidentiality of tokens [21], [24], [52], [7] (see Section IV-D for details).

Meanwhile, in each login the IdP issues an identity token that (a) specifies the target RP (i.e., *RP designation*) and (b) identifies the authenticated user who requests this token (i.e., *user identification*), as the enclosed (pseudo-)identities of RP and user [1], [2], [3] (see Section V-B for details). RP designation prevents malicious RPs from replaying received identity tokens to gain illegal access to other honest RPs as victim users. So an RP usually checks the RP (pseudo-)identity enclosed in a token before accepting it [1], [7], [6], [5], [11], [10], [13], [12] and then allows the token holder to log in as the specified (pseudo-)identity (i.e., account).

B. Identity Transformations in SSO

Existing privacy-preserving solutions of SSO hide only RP identities or user identities (i.e., only IdP-based login tracing or RP-based identity linkage is prevented), or introduce an extra fully-trusted component to process the (pseudo-)identities of both RP and user before an identity token is issued.

UPPRESSO *simultaneously* transforms the identities of RP and user in each login, without extra trusted components more than the honest-but-curious IdP, to provide untraceable and unlinkable SSO service. Table II lists the notations of the identity-transformation approach, and a sub/super-script (i.e., i , j , or l) may be omitted if it does not cause ambiguity.

First of all, each registered user owns her account unique at an RP. $\mathcal{F}_{Acct*}(ID_{U_i}, ID_{RP_j}) = Acct_{i,j}$ is proposed to determine a user's account at an RP. An account is *meaningful* at an RP if it belongs to some *registered* user at this RP, while a *meaningless* account A_m at RP_j does not belong to any registered user, i.e., $\forall ID_{U_i} \in \mathbb{ID}_U = \{ID_{U_i; 1 \leq i \leq s}\} : A_m \neq \mathcal{F}_{Acct*}(ID_{U_i}, ID_{RP_j})$. That is, meaningless accounts belongs to “potential” users but not registered yet.

To ensure IdP untraceability (or prevent IdP-based login tracing), a user initiates a login by calculating an *ephemeral* pseudo-identity $PID_{RP_j}^l$ for the target RP and sending an identity-token request for $PID_{RP_j}^l$ (but not ID_{RP_j}) to an IdP.

After authenticating the initiating user as ID_{U_i} , the IdP calculates an *ephemeral* $PID_{U_i,j}^l$ based on ID_{U_i} and $PID_{RP_j}^l$,

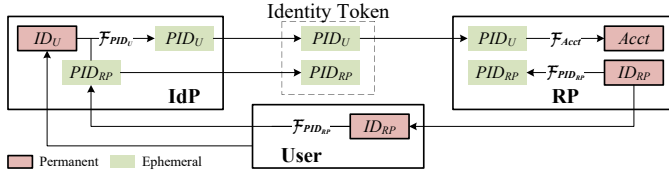


Fig. 2. Identity transformations in UPPRESSO

and then issues an identity token that binds $PID_{U,j}^l$ and $PID_{RP,j}^l$. To ensure RP unlinkability (i.e., prevent RP-based identity linkage), $ID_{U,i}$ is not enclosed in the identity token.

On receiving a signed token, the RP calculates the user's *permanent* account (i.e., $Acct_{i,j}$) based on $PID_{U,i,j}^l$ and $PID_{RP,j}^l$, and allows the token holder to log in as $Acct_{i,j}$.

Therefore, in addition to $\mathcal{F}_{Acct*}(ID_U, ID_{RP}) = Acct$, we propose the following identity transformations in a login initiated by a user to visit an RP in privacy-preserving SSO:

- $\mathcal{F}_{PID_{RP}}(ID_{RP}) = PID_{RP}$, calculated by the initiating user, where ID_{RP} is the target RP's identity. In the IdP's view, $\mathcal{F}_{PID_{RP}}()$ is a one-way function and PID_{RP} is *indistinguishable* from random variables.
- $\mathcal{F}_{PID_U}(ID_U, PID_{RP}) = PID_U$, calculated by the IdP, where ID_U identifies the user requesting tokens. In the target RP's view, $\mathcal{F}_{PID_U}()$ is a one-way function and PID_U is *indistinguishable* from random variables.
- $\mathcal{F}_{Acct}(PID_U, PID_{RP}) = Acct$, calculated by the target RP. In the user's multiple logins visiting the RP, $Acct = \mathcal{F}_{Acct*}(ID_U, ID_{RP})$ is always derived.

The relationships among the (pseudo-)identities are depicted in Figure 2. Red and green blocks represent *permanent* identities and *ephemeral* pseudo-identities, respectively, and labeled arrows denote the transformations of (pseudo-)identities. PID_{RP} and PID_U are ephemeral; otherwise, the IdP could learn the visited RP. $PID_{U,j}$ for visiting different RPs are independent of each other; otherwise, colluding RPs could learn something on ID_U from $PID_{U,j}$ s and link the logins.

In privacy-preserving SSO services integrating the identity transformations, an identity token *explicitly binds* $PID_{RP,j} = \mathcal{F}_{PID_{RP}}(ID_{RP,j})$ but *implicitly designates* only the RP with $ID_{RP,j}$. The following properties are required for the identity transformations:

- **Account Uniqueness.** Each user owns an account locally unique at an RP, determined by $\mathcal{F}_{Acct*}(ID_U, ID_{RP})$.
- **RP Designation.** An identity token is rejected, at any honest RPs other than the designated one.
- **User Identification.** From an identity token, the designated honest RP derives only the account exactly identifying the user requesting this token.
- **IdP Untraceability.** The IdP learns nothing on the visited RP from the identity-token requests.
- **RP Unlinkability.** Colluding RPs cannot link any login initiated by an honest user visiting an RP, to any logins visiting any other colluding RPs by honest users.

In addition to the basic requirement of account uniqueness, RP designation and user identification ensure security of SSO services, while IdP untraceability and RP unlinkability protect user privacy (see Section V for the formal proofs).

IV. THE DESIGNS OF UPPRESSO

A. Threat Model

The system consists of an honest-but-curious IdP, as well as several honest or malicious RPs and users. This model is in line with widely-deployed SSO services in the Internet [1], [2], [3], [4].

Honest-but-curious IdP. The IdP strictly follows the protocols, while remaining interested in learning about a user's login activities. For example, it could store received messages to infer the relationship among ID_U , ID_{RP} , PID_U , and PID_{RP} . It never actively violates the protocols, so a user's script downloaded from the IdP also follows the protocols.

The IdP maintains its private key well for signing identity tokens and RP certificates, and adversaries cannot forge such signed messages.

Malicious RPs. Adversaries could control a set of RPs by registering at the IdP as an RP or exploiting vulnerabilities to compromise some RPs. Malicious RPs could behave arbitrarily, attempting to break the security or privacy guarantees for honest users and RPs [6], [52], [11], [12].

Malicious users. Adversaries could control a set of users by stealing their credentials or registering Sybil users in the system. A malicious user could modify, insert, drop, or replay messages or even behave arbitrarily.

Colluding RPs and users. Malicious users and RPs could collude, attempting to break the security or privacy guarantees; that is, their objective is to (a) impersonate a victim user at honest RPs, (b) entice an honest user to log into an honest RP under another user's account, and (c) link an honest user's logins visiting colluding RPs. For example, they could manipulate PID_{RP} and t in a login, attempting to (a) entice honest users to return an identity token that could be accepted by some honest RP or (b) affect the generation of PID_U to analyze the relationship between ID_U and PID_U .

B. Assumptions

We assume secure communications between honest entities, and the cryptographic primitives are secure. The software stack of an honest entity (e.g., a browser) is correctly implemented to transmit messages to receivers as expected. These assumptions are realized by the commonly-used mechanisms in popular SSO services [1], [2], [3], [21], [24], [52], [7], [22] (e.g., HTTPS/TLS, HTTP session maintenance, and the `postMessage` HTML5 API in COTS browsers; see Section IV-D for specific designs for web applications).

As PID_{RP} but not ID_{RP} is processed by the IdP, it also requires that no other information about the visited RP is leaked to the IdP, in the requesting, receiving and forwarding of identity tokens. We realize this by specific designs in Section IV-D. Our work focuses on the privacy threats introduced by SSO protocols, and does not consider the tracking of user activities by network traffic analysis or crafted web pages, as they can be prevented by other defenses.

UPPRESSO is designed for users who value privacy, and provides an easy-to-deploy option for such users. So privacy leakages due to re-identification by distinctive attributes across RPs are out of our scope. A user never consents to enclose

distinctive attributes, such as telephone number, Email address, etc., in tokens or sets such attributes at any RP.

C. Identity-Transformation Algorithms

We propose the following identity transformations on an elliptic curve $\mathbb{E}$, where G is a generator on $\mathbb{E}$ of order n .

ID_U , ID_{RP} and $Acct$. The IdP assigns a unique random integer $u \in \mathbb{Z}_n$ to a registered user (i.e., $ID_U = u$), and randomly selects unique $ID_{RP} = [r]G$ for an RP. Here, $[r]G$ denotes the addition of G on the curve r times. Then, $Acct_{i,j} = \mathcal{F}_{Acct*}(ID_{U_i}, ID_{RP_j}) = [ID_{U_i}]ID_{RP_j} = [u_i r_j]G$ is automatically assigned to a registered user at every RP.

All entities except the IdP know nothing about a user's identity $ID_U = u$; otherwise, colluding RPs could calculate $[u]ID_{RP_j}$ for any known u and link these accounts. This requires that $|\mathbb{Z}_n|$ is sufficiently large against exhaustive attacks; for example, n is approximately 2^{256} for the NIST P256 elliptic curve and the amount of users is usually less than 2^{64} .

$ID_{RP} = [r]G$ and $Acct = [ID_U]ID_{RP}$ are publicly-known, but r is always kept secret; otherwise, two colluding RPs with $ID_{RP_j} = [r]G$ and $ID_{RP_{j'}} = [r']G$ could link a user's accounts by checking whether $[r']Acct_j$ is equal to $[r]Acct_{j'}$ or not.

In UPPRESSO r is processed only *once* by the IdP, while u is used only by the IdP *internally* and not enclosed in any message; see Section IV-E for details.

ID_{RP} - PID_{RP} Transformation. When visiting an RP, a user selects a random number $t \in \mathbb{Z}_n$ and calculates PID_{RP} as below. She also sends t to the RP.

$$PID_{RP} = \mathcal{F}_{PID_{RP}}(ID_{RP}, t) = [t]ID_{RP} \quad (1)$$

ID_U - PID_U Transformation. After receiving an identity-token request for PID_{RP} from a user authenticated as ID_U , the IdP calculates PID_U as below.

$$PID_U = \mathcal{F}_{PID_U}(ID_U, PID_{RP}) = [ID_U]PID_{RP} \quad (2)$$

PID_U - $Acct$ Transformation. After verifying a signed identity token, the RP calculates $Acct$ as follows.

$$Acct = \mathcal{F}_{Acct}(PID_U, t) = [t^{-1} \bmod n]PID_U \quad (3)$$

First of all, the above designs ensure *account uniqueness*: At an RP, every user owns a unique account, because $ID_{RP} = [r]G$ is also a generator of $\mathbb{E}$ and then $Acct = \mathcal{F}_{Acct*}() = [ur]G$ is unique locally at this RP.

In a login involving no adversary, the visited RP always derives the account belonging to the user who requests the identity token, by calculating $\mathcal{F}_{Acct}(PID_U, PID_{RP})$. From Equations 1, 2, and 3, it is derived that

$$Acct = [t^{-1}utr]G = [ur]G = \mathcal{F}_{Acct*}(ID_U, ID_{RP}) \quad (4)$$

In Section V we formally prove the other required properties (i.e., *RP designation*, *user identification*, *IdP untraceability*, and *RP unlinkability*) of the identity transformations under the adversarial scenarios.

D. The Designs Specific for Web Applications

When the required properties of identity transformations are satisfied, in UPPRESSO we need to additionally ensure:

1. $PID_{RP} = [t]ID_{RP}$ is calculated based on the target RP's identity but not any other RPs' (i.e., the target RP is designated),
2. An identity token is forwarded to only the target RP web server in a browser, as authenticity and integrity of tokens have been ensured by the IdP's signatures and they are transmitted over HTTPS/TLS,
3. t is known only to the target RP and the initiating user (but not the IdP; otherwise, the IdP is able to infer ID_{RP} from PID_{RP}), and no direct privacy leakage in the requesting of identity tokens (e.g., $ReEnptUrl_{RP}$ or ID_{RP} is not included in the requests, and no HTTP `referer` header about the RP web server is leaked in HTTP requests to the IdP).

In an SSO system implementing OIDC with pop-up UX, as shown in Figure I, a user requests a token by sending the target RP's identity (i.e., ID_{RP}) via the user-i window, and the IdP replies with the signed token and also the RP's URL to receive tokens (i.e., $ReEnptUrl_{RP}$). Then, after receiving the token, the user-i script forwards it to the user-r script by `postMessage` which is restricted by the origin of $ReEnptUrl_{RP}$ [6], [53], [7], [21], [24], [1]. The `postMessage` `targetOrigin` mechanism [23] restricts the recipient, so the token is forwarded to exactly the target RP because (a) only scripts downloaded from the origin of $ReEnptUrl_{RP}$ are legitimate recipients of the forwarded tokens and (b) the relationship between $ReEnptUrl_{RP}$ and ID_{RP} is maintained at the honest IdP server.

The following improvements are proposed in UPPRESSO, to satisfy the additional requirements in an OIDC-compatible way. First of all, the IdP receives PID_{RP} (but not ID_{RP}) in an identity-token request, so it cannot reply with $ReEnptUrl_{RP}$ to the user-i window. An RP certificate, denoted as $Cert_{RP}$, is introduced to bind $ReEnptUrl_{RP}$ and ID_{RP} , signed by the IdP during the RP's initial registration. Then, in each login, as shown in Figure 3, when the user-i window is popped up, $Cert_{RP}$ and the scope of request user attributes are processed with the HTTP request to the IdP, all of which are generated by the user-r script downloaded from the target RP. This HTTP request downloads the user-i script, which in turn verifies $Cert_{RP}$ to extract ID_{RP} and $ReEnptUrl_{RP}$, and calculates $PID_{RP} = [t]ID_{RP}$. Then, after the IdP issues a token, the user-i script forwards it to the user-r script by `postMessage` which is restricted by the origin of $ReEnptUrl_{RP}$. Besides, on receiving an identity-token request, the IdP web server checks the included HTTP `origin` header to ensure it is sent by the user-i script, which will honestly verify $Cert_{RP}$ to ensure the correct relationship between $ReEnptUrl_{RP}$ and ID_{RP} . This ensures that $PID_{RP} = [t]ID_{RP}$ is calculated based on the visited RP's identity and only the designated RP receives the token, because (a) $ReEnptUrl_{RP}$ and ID_{RP} are signed in $Cert_{RP}$ and (b) only scripts from the origin of $ReEnptUrl_{RP}$ are legitimate recipients of `postMessage`.

In the HTTP request to pop up the user-i window, $Cert_{RP}$ and the scope of requested user attributes are carried and processed as URL *fragments* (but not query parameters) [54], which are identified by # instead of ?, so that they are not sent to the IdP web server but locally processed by the browser. Meanwhile, we save t and $ReEnptUrl_{RP}$ using the `sessionStorage` HTML object [55], because the user-i window may be refreshed to finish user authentication and attribute consent. That is, t , $ReEnptUrl_{RP}$ and $Cert_{RP}$ are processed within browsers, not sent to the IdP.

To prevent referer leakage in the HTTP request to pop up the user-i window, we need to ensure this request does not carry an HTTP `referer` header, which reveals the visited RP's domain to the IdP web server.³ It is achieved by setting `<meta name="referrer" content="no-referrer">` in the RP's SSO login page [56].

Finally, two endpoints specified in OIDC [2], [1] for the implicit login flow are implemented in an OIDC-compatible way. When receiving a token request for an "unregistered" ID_{RP} (i.e., ephemeral PID_{RP}), the IdP issues an identity token to an "empty" $ReEnptUrl_{RP}$, and the calculation of PID_U is considered as a special way to assign PPIDs. The RP's redirection endpoint is also implemented compatibly, where t is sent along with the tokens as an optional parameter. Moreover, the IdP's public key is set in the user-i script to verify $Cert_{RP}$, so a user configures nothing locally, like in other OIDC implementations [1], [21], [24], [22].

E. The UPPRESSO protocol

System Initialization. $\mathbb{E}$, G and n are set up and publicly published. An IdP generates a key pair (SK, PK) to sign and verify identity tokens and RP certificates.

RP Registration. Each RP registers itself at the IdP to obtain ID_{RP} and its RP certificate $Cert_{RP}$ as follows. This may be conducted by face-to-face or online [57] means.

1. An RP pre-installs PK by trusted means. It submits a registration request, including the URL to receive identity tokens (i.e., $ReEnptUrl_{RP}$) and other information.
2. After examining the request, the IdP randomly selects $r \in \mathbb{Z}_n$ and assigns a *unique* point $[r]G$ to the RP as its identity. Note that r is not processed any more and then known to nobody due to the elliptic curve discrete logarithm problem (ECDLP). The IdP then signs $Cert_{RP} = [ID_{RP}, ReEnptUrl_{RP}, *]_{SK}$, where $[\cdot]_{SK}$ is a message signed using SK and $*$ is supplementary information such as the RP's unambiguous name.
3. The RP verifies $Cert_{RP}$ using PK , and accepts ID_{RP} and $Cert_{RP}$.

User Registration. Each user sets up her unique username and the corresponding credential for the IdP. The IdP assigns a random but unique identity $ID_U = u \in \mathbb{Z}_n$ to the user.

ID_U is known *only* to the IdP and other entities know nothing about ID_U . ID_U is used only by the IdP *internally* in UPPRESSO, not enclosed in any message. For example, a user's identity is generated and always restored by hashing

her username concatenated with the IdP's private key. Then, ID_U is never stored in hard disks, and protected almost the same as the IdP's private key because it is *only* used to calculate PID_U as the IdP is signing an identity token binding $PID_U = [ID_U]PID_{RP}$.

SSO Login. A login flow involves three steps: Identity-token requesting, generation, and acceptance. In Figure 3, the IdP's and RP's operations are connected by two vertical lines, respectively. The user operations are split into two groups in different browser windows by vertical lines, one communicating with the IdP and the other with the RP. Solid horizontal lines indicate messages exchanged between a user and the IdP (or the RP), while dotted lines represent the mechanisms within a browser: (a) `postMessage` invocations between two scripts (or browser windows) or (b) `sessionStorage` objects accessed by refreshed pages of an HTTP session.

1. *Identity-Token Requesting.* The user requests an identity token for $PID_{RP} = [t]ID_{RP}$.

- 1.1 The target RP provides an SSO login page with the user-i script, and $Cert_{RP}$ is set in this script. The user clicks the SSO login button, to pop up another browser window responsible for communicating with the IdP.
- 1.2 The user-i window downloads the user-i script, which processes $Cert_{RP}$ and the scope of requested attributes sent from the user-r window. It verifies $Cert_{RP}$, extracts ID_{RP} and $ReEnptUrl_{RP}$ from $Cert_{RP}$, chooses a random number $t \in \mathbb{Z}_n$, and calculates $PID_{RP} = [t]ID_{RP}$.
- 1.3 Then, the user-i script saves $ReEnptUrl_{RP}$ and t in the `sessionStorage` HTML5 object, and requests an identity token from the IdP by sending PID_{RP} and the scope of requested attributes.

2. *Identity-Token Generation.* The IdP calculates $PID_U = [ID_U]PID_{RP}$ and signs an identity token as follows.

- 2.1 On receiving an identity-token request, the IdP authenticates the initiating user as ID_U , if not authenticated yet. Then, it obtains the user's consent to enclose attributes.
- 2.2 The IdP calculates $PID_U = [ID_U]PID_{RP}$, and signs $TK = [PID_{RP}, PID_U, Iss, Validity, Attr]_{SK}$, where Iss identifies the IdP, $Validity$ indicates the validity period, and $Attr$ contains the user attributes.
- 2.3 The IdP replies with the identity token to the user-i window, as well as the user-i script to forward this token.

3. *Identity-Token Acceptance.* The RP receives the identity token and allows the user to log in.

- 3.1 From the `sessionStorage` object the user-i script loads $ReEnptUrl_{RP}$ and t , and forwards the identity token and t to the user-r window by `postMessage` which is restricted by the origin of $ReEnptUrl_{RP}$.
- 3.2 The user-r script submits it to the RP. The RP verifies the signature and the validity period of the received token, and then calculates $Acct = [t^{-1}]PID_U$. Only if $Acct$ is meaningful, the RP allows the user to log in.

If any verification fails, the flow will be immediately terminated. For instance, when receiving invalid $Cert_{RP}$ the user halts it. The IdP rejects an identity-token request if the received PID_{RP} is not a point on $\mathbb{E}$, and the RP rejects a token if it is not well signed or a meaningless account is derived.

³In original OIDC services this privacy leakage commonly exists, but does not matter because such services do not aim to prevent IdP-based login tracing.

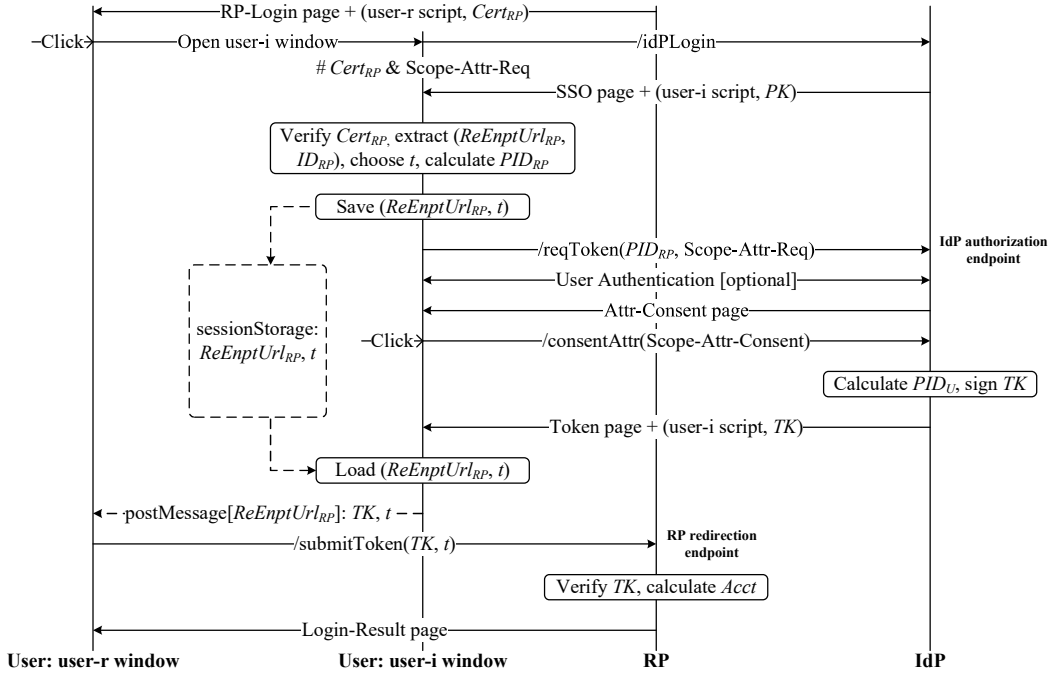


Fig. 3. The SSO login flow of UPPRESSO

In Step 3.2 an RP does not check whether PID_{RP} in the token is equal to $[t]ID_{RP}$ or not, and this efficient design does not result in attacks, which is proved in Section V-B.

In order to distinguish meaningless accounts from meaningful ones in Step 3.2, it requires that the RP *locally* maintains an up-to-date account set, i.e., $\{Acct_i\}_j = \{[u_{i:1 \leq i \leq s} r_j]G\}$ for all registered users and $|\{Acct_i\}_j| = s$. We discuss the means of account synchronization in Section V-E.

V. SECURITY AND PRIVACY ANALYSIS

As account uniqueness is analyzed in Section IV-C this section proves security (i.e., RP designation and user identification) and privacy (i.e., IdP untraceability and RP unlinkability) of the identity transformations in UPPRESSO.

A. Adversarial Scenarios

According to our design goals and the potential adversaries listed in Section IV-A, we consider three adversarial scenarios as below and the security and privacy guarantees are proved against different adversaries.

First of all, security (i.e., RP designation and user identification) is proved with an honest IdP but against *malicious RPs and users*. In the proofs, malicious entities could collude [52], [7], [6], attempting to (a) impersonate an honest user to log into some honest RP or (b) entice an honest user to log into an honest RP under another's account, by arbitrarily manipulating t and/or sending TK to an RP not designated.

Provided that the security properties are satisfied, we analyze the privacy properties *only* for successful logins. It makes no sense to track user activities from malicious behaviors deviating from the protocols, resulting unsuccessful logins. So no malicious entities are considered in the proofs of privacy. In particular, (a) IdP untraceability is proved in Section V-C

against an *honest-but-curious IdP*, which tries to infer the identities of the *honest* RPs being visited by an *honest* user, and (b) RP unlinkability is analyzed in Section V-D against a *group of semi-honest RPs and users* which follow the protocols but share their information, attempting to link an honest user's accounts across these semi-honest RPs.

B. Security

In secure SSO systems, an identity token denoted as TK , which is requested by a user to visit an RP, *enables only this user to log into only the honest target RP as her account at this RP* (i.e., RP designation and user identification). It makes no sense to discuss the login results at malicious RPs.

Let's assume totally s users and p RPs in UPPRESSO, whose identities are denoted as $\mathbb{ID}_U = \mathbb{u} = \{u_{i:1 \leq i \leq s}\}$ and $\mathbb{ID}_{RP} = \{[r_{j:1 \leq j \leq p}]G\}$, respectively. $Acct_{i,j}$ and ID_{RP_j} are publicly-known, while u_i and r_j are kept secret to adversaries.

TK is signed by the IdP to bind PID_{RP} and PID_U , where (a) the RP with ID_{RP} is designated if $PID_{RP} = [t]ID_{RP}$ is calculated and (b) $PID_U = [ID_U]PID_{RP}$ is calculated based on the initiating user's identity ID_U .

RP designation and user identification are proved in Theorems 1 and 2, respectively, against malicious RPs colluding with users. Besides, as mentioned in Section IV-E, in Step 3.2 an (honest) RP directly derives accounts from any signed and unexpired identity token, without checking whether PID_{RP} in the token is equal to $[t]ID_{RP}$ or not. So in the proofs we do not assume any relationship among t , ID_{RP} and PID_{RP} which are received by an RP.

RP designation means that, *from TK only the designated honest RP derives meaningful accounts*. We prove this by showing that, at any honest RPs other than the designated one, no meaningful account will be derived from TK .

THEOREM 1 (RP Designation): When given known $\mathbb{ID}_{RP}$, known $\mathcal{A}cct$ and unknown $\mathbb{ID}_U$ defined as above, malicious adversaries cannot find j_1, j_2, i_1, i_2, t_1 , and t_2 which satisfy that $\mathcal{F}_{Acct}(\mathcal{F}_{PID_U}(ID_{U_{i_1}}, PID_{RP}^*), t_2) = \mathcal{F}_{Acct^*}(ID_{U_{i_2}}, ID_{RP_{j_2}})$, where PID_{RP}^* may be calculated as $\mathcal{F}_{PID_{RP}}(ID_{RP_{j_1}}, t_1)$ (i.e., RP_{j_1} is designated) or even arbitrarily generated (i.e., no RP is designated), $j_1 \neq j_2, 1 \leq j_1, j_2 \leq p$, and $1 \leq i_1, i_2 \leq s$.

PROOF. If adversaries could find j_1, j_2, i_1, i_2, t_1 and t_2 which satisfy that $[t_2^{-1}t_1ID_{U_{i_1}}]ID_{RP_{j_1}} = [ID_{U_{i_2}}]ID_{RP_{j_2}}$ where $j_1 \neq j_2$, the ECDLP of $t_2^{-1}t_1 = \log_{[u_{i_1}r_{j_1}]G}[u_{i_2}r_{j_2}]G = \log_{Acct_{i_1,j_1}}Acct_{i_2,j_2}$ would be solved.

The adversary might alternatively attempt to find PID_{RP}^* , j_2, i_1, i_2 and t_2 which satisfy that $[t_2^{-1}ID_{U_{i_1}}]PID_{RP}^* = [ID_{U_{i_2}}]ID_{RP_{j_2}}$, but this cannot be solved due to unknown ID_{U_i} when $i_1 \neq i_2$. Or, if $i_1 = i_2$, it would require the adversary to solve $[t_2^{-1}]PID_{RP}^* = ID_{RP_{j_2}}$ or $PID_{RP}^* = [t_2]ID_{RP_{j_2}}$. That is, the adversary chooses t_2 , calculates $PID_{RP}^* = [t_2]ID_{RP_{j_2}}$, requests a token for PID_{RP}^* , and then logs into RP_{j_2} to “impersonate” himself. This is not an attack: RP_{j_2} is actually designated by TK and the adversary’s account is derived at RP_{j_2} . $\square$

User identification means that, from TK the designated honest RP derives only the meaningful account belonging to the user requesting TK . We prove this property by showing that, at the RP designated by any TK , a meaningless account not belonging to the initiating user will never be derived.

THEOREM 2 (User Identification): Given unknown $\mathbb{ID}_U$ and known $\mathcal{A}cct$, for any known $ID_{RP} \in \mathbb{ID}_{RP}$, malicious adversaries cannot find $\hat{t}$ and TK binding $PID_{RP} = \mathcal{F}_{PID_{RP}}(ID_{RP}, \hat{t})$ and $PID_{\hat{U}} = \mathcal{F}_{PID_U}(ID_{\hat{U}}, PID_{RP})$ satisfying that $\mathcal{F}_{Acct}(PID_{\hat{U}}, \hat{t}) = \mathcal{F}_{Acct^*}(ID_{\hat{U}}, ID_{RP})$, where $ID_{\hat{U}} \neq ID_{\hat{U}}$ and $ID_{\hat{U}}, ID_{\hat{U}} \in \mathbb{ID}_U$.

PROOF. It requires that, for any ID_{RP} , adversaries cannot find $\hat{t}$ and $\hat{t}$ satisfying that $\mathcal{F}_{Acct}(PID_{\hat{U}}, \hat{t}) = [\hat{t}^{-1}]PID_{\hat{U}} = [\hat{t}^{-1}ID_{\hat{U}}]PID_{RP} = [\hat{t}^{-1}\hat{t}ID_{\hat{U}}]ID_{RP} = [ID_{\hat{U}}]ID_{RP} = \mathcal{F}_{Acct^*}(ID_{\hat{U}}, ID_{RP})$, where $ID_{\hat{U}} \neq ID_{\hat{U}}$.

When u_i and r_j are kept unknown, this is equivalent to the elliptic-curve discrete logarithm problem (ECDLP) of $\hat{t}^{-1}\hat{t} = \log_{[\hat{u}r]}G[\hat{u}r]G = \log_{\hat{Acct}}\hat{Acct}$. $\square$

C. Privacy against IdP-based Login Tracing

The honest IdP would attempt to infer the visited RPs, by collecting information from the identity-token requests. According to the designs and the detailed protocol in Sections IV-D and IV-E, it does not obtain any information about the target RP in a login (e.g., ID_{RP} , $ReEnptUtl_{RP}$ or $Cert_{RP}$), except the RP’s ephemeral pseudo-identity PID_{RP} .

We prove that, PID_{RP} is indistinguishable from a random variable on $\mathbb{E}$ to the IdP, and then it cannot (a) link multiple logins visiting an RP or (b) distinguish a login initiated by a user to visit any RP from those logins visiting other RPs by this user. Therefore, IdP untraceability is ensured.

THEOREM 3 (IdP Untraceability): The honest-but-curious IdP cannot distinguish $PID_{RP} = [t]ID_{RP}$ from a random variable on $\mathbb{E}$, where t is random in $\mathbb{Z}_n$ and kept unknown to the IdP.

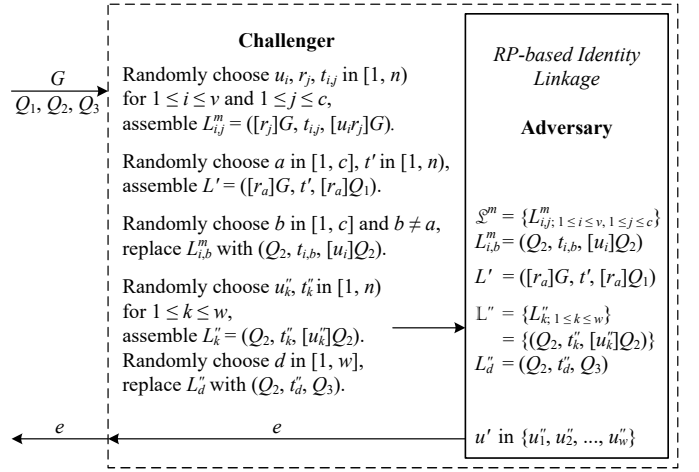


Fig. 4. The PPT algorithm $\mathcal{D}_R^*$ constructed based on the RP-based identity linkage game to solve the ECDDH problem.

PROOF. As G is a generator on $\mathbb{E}$ of order n , $ID_{RP} = [r]G$ is also a generator of order n . As t is uniformly-random in $\mathbb{Z}_n$ and kept unknown, $PID_{RP} = [t]ID_{RP}$ is indistinguishable from a point that is uniformly-random on $\mathbb{E}$. $\square$

This property actually has been proved in the Hash(EC)DH OPRF [39], [40] as obliviousness, where the OPRF server works similarly to the IdP and learns nothing about a client’s inputs or outputs of the evaluated pseudo-random function (i.e., ID_{RP} or $\mathcal{A}cct$ in UPPRESSO).

D. Privacy against RP-based Identity Linkage

Semi-honest RPs and users share their information, attempting to link an honest user’s accounts across the semi-honest RPs. In each login, an RP obtains *only* a random number t and an identity token enclosing PID_{RP} and PID_U . From the token, it learns only an ephemeral pseudo-identity $PID_U = [ID_U]PID_{RP}$ of the initiating user, from which it derives a *permanent* locally-unique identifier (or account) $\mathcal{A}cct = [ID_U]ID_{RP}$. It cannot directly calculate ID_U from PID_U or $\mathcal{A}cct$ due to the ECDLP.

Given the trapdoor t , adversaries can transform PID_{RP} and PID_U into ID_{RP} and $\mathcal{A}cct$, respectively, and vice versa. So we denote all the information that an RP learns in a login as a tuple $L = (ID_{RP}, t, \mathcal{A}cct) = ([r]G, t, [ur]G)$.

When c RPs share information with each other, they create a shared view of all logins visiting any of them, denoted as $\mathbb{L}$. When they further collect information from v semi-honest users, the logins initiated by these users are picked out of $\mathbb{L}$

and linked together as $\mathcal{L}^m = \begin{Bmatrix} L_{1,1}^m & L_{1,2}^m & \cdots & L_{1,c}^m \\ L_{2,1}^m & L_{2,2}^m & \cdots & L_{2,c}^m \\ \cdots & \cdots & L_{i,j}^m & \cdots \\ L_{v,1}^m & L_{v,2}^m & \cdots & L_{v,c}^m \end{Bmatrix}$,

where $L_{i,j}^m = ([r_j]G, t_{i,j}, [u_i r_j]G) \in \mathbb{L}$ for $1 \leq i \leq v$ and $1 \leq j \leq c$. Any login in $\mathbb{L}$ but not linked in $\mathcal{L}^m$ is initiated by an honest user to visit one of the c semi-honest RPs.

Next, in Theorem 4 we prove that, even when a number of semi-honest users share information with them, the semi-honest RPs still cannot link any login visiting a semi-honest

RP by an honest user to any other logins visiting other semi-honest RPs by honest users. Then, RP unlinkability is ensured.

THEOREM 4 (RP Unlinkability): Given $\mathbb{L}$ and $\mathcal{L}^m$, c semi-honest RPs and v semi-honest users cannot link any login visiting some of these RPs by an honest user to any subset of logins visiting other RPs by honest users.

PROOF. Out of $\mathbb{L}$ we randomly choose a login $L' \neq L_{i,j}^m$ ($1 \leq i \leq v, 1 \leq j \leq c$), which is initiated by an (unknown) honest user with $ID_{U'} = u'$ to a semi-honest RP_a where $a \in [1, c]$. Then, we randomly choose another semi-honest RP_b , where $b \in [1, c]$ and $b \neq a$. Consider any subset $\mathbb{L}'' \subset \mathbb{L}$ of w ($1 \leq w < s - v$) logins visiting RP_b by unknown honest users, and denote the identities of the users initiating these logins as $\mathbb{U}_w = \{u''_1, u''_2, \dots, u''_w\} \subset \mathbb{U}$.

Next, we prove that the group of semi-honest adversaries cannot distinguish $u' \in \mathbb{U}_w$ from u' randomly selected in the universal set (i.e., $u' \in \mathbb{Z}_n$). So the adversaries cannot link L' to another login (or any subset of logins) visiting RP_b .

We define an RP-based identity linkage game $\mathcal{G}_R$ between an adversary and a challenger, to describe this login linkage: the adversary receives $\mathcal{L}^m$, L' , and $\mathbb{L}''$ from the challenger and outputs e , where (a) $e = 1$ if it decides u' is in $\mathbb{U}_w$ or (b) $e = 0$ if it believes u' is randomly chosen from $\mathbb{Z}_n$. Thus, the adversary succeeds in $\mathcal{G}_R$ with an advantage $\mathbf{Adv}$:

$$\begin{aligned} \Pr_1 &= \Pr(\mathcal{G}_R(\mathcal{L}^m, L', \mathbb{L}'') = 1 \mid u' \in \mathbb{U}_w) \\ \Pr_2 &= \Pr(\mathcal{G}_R(\mathcal{L}^m, L', \mathbb{L}'') = 1 \mid u' \in \mathbb{Z}_n) \\ \mathbf{Adv} &= |\Pr_1 - \Pr_2| \end{aligned}$$

As depicted in Figure 4, we design a PPT algorithm $\mathcal{D}_R^*$ based on $\mathcal{G}_R$ to solve the elliptic curve decisional Diffie-Hellman (ECDDH) problem: Given $(G, [x]G, [y]G, [z]G)$, decide whether z is equal to xy or randomly chosen in $\mathbb{Z}_n$, where G is a point on an elliptic curve $\mathbb{E}$ of order n , and x and y are integers randomly and independently chosen in $\mathbb{Z}_n$.

The algorithm $\mathcal{D}_R^*$ works as follows. (1) On receiving an input $(G, Q_1 = [x]G, Q_2 = [y]G, Q_3 = [z]G)$, the challenger chooses random numbers in $\mathbb{Z}_n$ to construct $\{u_i\}$, $\{r_j\}$, and $\{t_{i,j}\}$ for $1 \leq i \leq v$ and $1 \leq j \leq c$, with which it assembles $L_{i,j}^m = ([r_j]G, t_{i,j}, [u_i r_j]G)$. It ensures $[r_j]G \neq Q_2$ (or $r_j \neq y$) in this procedure. (2) It randomly chooses $a \in [1, c]$ and $t' \in \mathbb{Z}_n$, to assemble $L' = ([r_a]G, t', [r_a]Q_1) = ([r_a]G, t', [x r_a]G)$. (3) Next, the challenger randomly chooses $b \in [1, c]$ but $b \neq a$, and replaces ID_{RP_b} with $Q_2 = [y]G$. Hence, for $1 \leq i \leq v$, the challenger replaces $L_{i,b}^m = ([r_b]G, t_{i,b}, [u_i r_b]G)$ with $(Q_2, t_{i,b}, [u_i]Q_2) = ([y]G, t_{i,b}, [u_i y]G)$, and finally constructs $\mathcal{L}^m$. (4) The challenger chooses random numbers in $\mathbb{Z}_n$ to construct $\{u''_k\}$ and $\{t''_k\}$ for $1 \leq k \leq w$, with which it assembles $\mathbb{L}'' = \{L''_{k;1 \leq k \leq w}\} = \{(Q_2, t''_k, [u''_k]Q_2)\} = \{([y]G, t''_k, [u''_k y]G)\}$. It ensures $[u''_k]G \neq Q_1$ (i.e., $u''_k \neq x$) and $u''_k \neq u_i$, for $1 \leq i \leq v$ and $1 \leq k \leq w$. Finally, it randomly chooses $d \in [1, w]$ and replaces L''_d with $(Q_2, t''_d, Q_3) = ([y]G, t''_d, [z]G)$. Thus, $\mathbb{L}'' = \{L''_{k;1 \leq k \leq w}\}$ represents the logins initiated by w honest users, i.e., $\mathbb{U}_w = \{u''_1, u''_2, \dots, u''_{d-1}, z/y, u''_{d+1}, \dots, u''_w\}$. (5) When the adversary of $\mathcal{G}_R$ receives $\mathcal{L}^m$, L' , and $\mathbb{L}''$ from the challenger, it returns e which is also the output of $\mathcal{D}_R^*$.

According to the above construction, x is embedded as $ID_{U'}$ of the login L' visiting the RP with $ID_{RP_a} = [r_a]G$, and z/y is embedded as $ID_{U''}$ of $\mathbb{L}''$ visiting the RP with $ID_{RP_b} = [y]G$, together with $\{u''_1, \dots, u''_{d-1}, u''_{d+1}, \dots, u''_w\}$. Meanwhile, $[r_a]G$ and $[y]G$ are two semi-honest RPs' identities in $\mathcal{L}^m$. Because $x \neq u''_{k;1 \leq k \leq w, k \neq d}$ and then x is not in $\{u''_1, \dots, u''_{d-1}, u''_{d+1}, \dots, u''_w\}$, the adversary outputs $s = 1$ and succeeds in this game only if $x = z/y$. Therefore, using $\mathcal{D}_R^*$ to solve the ECDDH problem, we have an advantage $\mathbf{Adv}^* = |\Pr_1^* - \Pr_2^*|$, where

$$\begin{aligned} \Pr_1^* &= \Pr(\mathcal{D}_R^*(G, [x]G, [y]G, [xy]G) = 1) \\ &= \Pr(\mathcal{G}_R(\mathcal{L}^m, L', \mathbb{L}'') = 1 \mid u' \in \mathbb{U}_w) = \Pr_1 \\ \Pr_2^* &= \Pr(\mathcal{D}_R^*(G, [x]G, [y]G, [z]G) = 1) \\ &= \Pr(\mathcal{G}_R(\mathcal{L}^m, L', \mathbb{L}'') = 1 \mid u' \in \mathbb{Z}_n) = \Pr_2 \\ \mathbf{Adv}^* &= |\Pr_1^* - \Pr_2^*| = |\Pr_1 - \Pr_2| = \mathbf{Adv} \end{aligned}$$

If the adversary of $\mathcal{G}_R$ has a non-negligible advantage, then $\mathbf{Adv}^* = \mathbf{Adv}$ is also non-negligible regardless of the security parameter λ . This violates the ECDDH assumption.

So the adversary has no advantage in $\mathcal{G}_R$ and cannot decide whether L' is initiated by some honest user with an identity in $\mathbb{U}_w$ or not. Because RP_b is any semi-honest RP, this proof can be extended from RP_b to more semi-honest RPs. $\square$

E. Account Synchronization

To distinguish meaningless accounts from meaningful ones in Step 3.2, an RP needs to synchronize the accounts from the IdP to maintain an *up-to-date* set of meaningful accounts; otherwise, a *newly-registered* user will be rejected because the derived account is *falsely* considered as meaningless (i.e., not in the local set).

Naive account synchronization may be finished as below. On receiving an identity token from which a meaningless account is derived, an RP instantly contacts the IdP to retrieve the account set of users registered after the RP's last synchronization. Then, after this instant account synchronization,⁴ the token holder will be rejected if the derived account is still not in the updated set.

We propose the following "silent" account synchronization leveraging *conditional* PID_{RP} checking as below, eliminating the communications in the "active" synchronization as above. When an RP derives some account not in its local account set, it additionally checks PID_{RP} in this case: If PID_{RP} enclosed in the signed token is equal to $[t]ID_{RP}$, this account is asserted to belong to a newly-registered user (i.e., meaningful) and then the RP locally updates its account set (i.e., this derived account is inserted into the set); otherwise, it rejects this token binding $PID_{RP} \neq [t]ID_{RP}$. In this case, adversaries cannot (a) manipulate t to visit the designated RP

⁴An RP does not straightforwardly treat the token holder with an account not in its local set as a newly-registered user; otherwise, although a malicious user cannot log into this RP as any meaningful account belonging to others according to Theorem 2, she could log into such an RP as an meaningless but identical account in multiple logins. For example, by choosing $(\hat{t}_1, \hat{t}_1)$ and $(\hat{t}_2, \hat{t}_2)$ in two logins where $m = \hat{t}_1/\hat{t}_1 = \hat{t}_2/\hat{t}_2$, as analyzed in the proof of Theorem 2, this malicious user would actually own another account $Acct = [mID_U]ID_{RP} \neq [ID_U]ID_{RP}$.

because t is checked in the equation of $PID_{RP} = [t]ID_{RP}$ or (b) induce any RP other than the designated one to accept this token because they cannot solve t' satisfying that $[t']ID_{RP'} = [t]ID_{RP}$, which is equivalent to solve the ECDLP. So the derived account must be meaningful when PID_{RP} is equal to $[t]ID_{RP}$.

VI. IMPLEMENTATION AND EVALUATION

We implemented the UPPRESSO prototype⁵ and conducted experimental comparisons with two open-source SSO systems: (a) MITREid Connect [53], a PPID-enhanced OIDC system with redirect UX, preventing only RP-based identity linkage, and (b) SPRESSO [6], which prevents only IdP-based login tracing. All these solutions work with COTS browsers as user agents, while UPPRESSO and SPRESSO implement OIDC-compatible services with pop-up UX.

A. Prototype Implementation

The UPPRESSO prototype implemented the identity transformations on the NIST P256 elliptic curve where $n \approx 2^{256}$, and the IdP was developed on top of MITREid Connect [53], with minimal code modifications. All systems employ RSA-2048 and SHA-256 to generate tokens. The scripts of user-i and user-r consist of about 180 and 80 lines of JavaScript code, respectively. The cryptographic computations such as $Cert_{RP}$ verification and PID_{RP} negotiation are conducted based on elliptic [58], an open-source JavaScript library.

We developed a Java-based RP SDK with about 310 lines of code on the Spring Boot framework. The main function encapsulates the RP operations of UPPRESSO: verify an identity token and derive $Acct$. The cryptographic computations are finished using the Spring Security library. Then, an RP can invoke necessary functions by adding less than 10 lines of Java code, to access the services provided by UPPRESSO.

SPRESSO implements all entities by JavaScript based on node.js, while MITREid Connect provides Java implementations of IdP and RP SDK. Thus, in UPPRESSO and MITREid Connect, we implemented RPs based on Spring Boot by integrating the respective SDKs. In all three schemes, the RPs provide the same function of obtaining the user's account from a verified identity token.

B. Performance Evaluation

Experiment setting. We conducted experiments in two settings: (a) on a PC with AMD Ryzen 7 5700X CPU and 32GB memory, a browser, the IdP, the visited RP, and the extra forwarder server of SPRESSO were deployed on separated virtual machines, each of which ran Windows Server 2025 with 2 vCPUs and 4GB RAM, and (b) the browser running locally on the PC with AMD Ryzen 7 5700X CPU and 32GB memory, remotely accessed the servers (IdP and RP servers, and the forwarder server of SPRESSO) deployed on the Alibaba Cloud Elastic Compute Service, each of which ran Windows Server 2025 with 8 vCPUs and 32GB RAM. The first setting ran all functions on separated virtual machines

but within the same physical PC, to minimize the impact of network delays.

Comparisons. We split the login flow into three phases for detailed comparisons: (1) *identity-token requesting* (Step 1 in Figure 3), to construct an identity-token request and send it to the IdP server; (2) *identity-token generation* (Step 2), to generate an identity token at the IdP server, while the user authentication and the user-attribute consent are excluded; and (3) *identity-token acceptance* (Step 3), where the RP receives, verifies, and parses the identity token.

The IdP and the RP of all solutions, as well as the forwarder server in SPRESSO, require user-side scripts, which introduces additional time for downloading. In the identity-token requesting phase of UPPRESSO a browser downloads the user-i script, as described in Section IV-D. As mentioned in Section II-A, to process the token retrieved from the IdP, in MITREid Connect a user-r script is downloaded during the phase of token generation. In addition to a script from the IdP in identity-token requesting, SPRESSO needs another script from the forwarder server in the token acceptance phase [6]. Fortunately, once a browser visits a web server, it caches the scripts locally, reducing the download time for subsequent visits. This caching is enabled by the `localStorage` HTML object [55] and HTTP `Cache-Control` header.

We measured the time cost for both initial and subsequent visits (1,000 trials) and compared the average SSO login times of three schemes. As shown in Figure 5, MITREid Connect, UPPRESSO, and SPRESSO require (a) 74 ms, 187 ms, and 282 ms for initial visits, and 69 ms, 158 ms and 277 ms for subsequent visits, or (b) 222 ms, 372 ms, and 605 ms for initial visits, and 219 ms, 318 ms and 576 ms for subsequent visits.

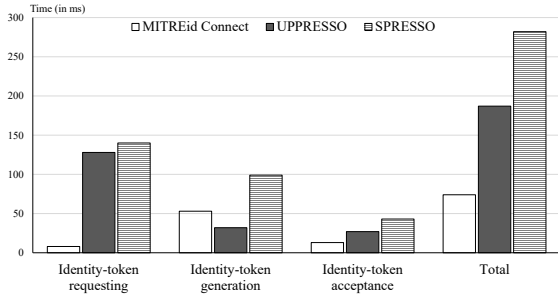
Regarding identity-token requesting, the RP of MITREid Connect immediately constructs an identity-token request. UPPRESSO incurs overheads in opening a new browser window and downloading the user-i script. In SPRESSO the user also needs to open a new browser window, and obtain information about the IdP and encrypt the visited RP's domain using an ephemeral key, resulting in extra overheads.

UPPRESSO requires the least time for generating identity tokens for it receives the token from the IdP without additional processing. In this phase MITREid Connect and SPRESSO require extra time as the browser downloads a script from the RP and the forwarder, respectively. After receiving an identity token from the IdP, in SPRESSO the user decrypts the RP's domain by scripts and sends it to the RP's redirection endpoint. Moreover, SPRESSO takes slightly more time to generate an identity token, as it implements the IdP using node.js and uses a JavaScript cryptographic library that is a little less efficient than the Java library used in the others.

Finally, in the identity-token acceptance phase, three solutions take similar amounts of time for the RP to receive a token and accept it.

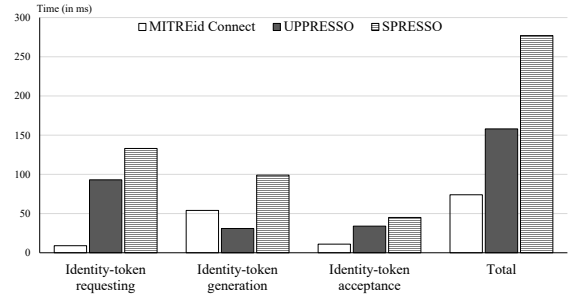
By comparing the average time costs of initial visits and subsequent visits, we find that the performances of all schemes are improved and the improvement is the most significant in UPPRESSO for the size of its scripts is the largest (134kB vs. 2.34kB in MITREid Connect and 6.84kB in SPRESSO).

⁵The prototype is open-sourced at <https://github.com/uppresso/>.

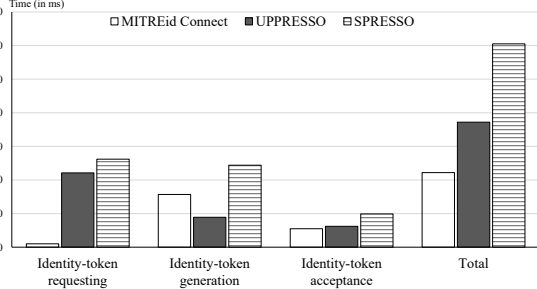


[i] Initial visit

(a) Within a local virtual environment

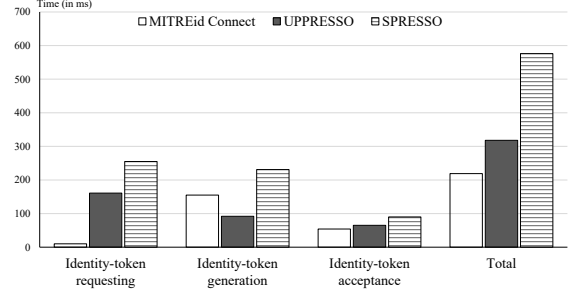


[ii] Subsequent visit



[i] Initial visit

(b) With a remotely-visiting browser



[ii] Subsequent visit

Fig. 5. The time costs of SSO login in MITREid Connect, UPPRESSO, and SPRESSO

VII. DISCUSSIONS

IdP-RP collusive attacks. This work prefers to provide better user convenience by easy-to-deploy privacy-preserving designs, rather than prevent IdP-RP collusive attacks. Unless a user secret is involved in the derivation of accounts, SSO cannot protect user privacy against an IdP colluding with RPs. For example, even when the IdP strictly follows the protocols but shares information with RPs, a user would complete her logins *entirely* with colluding entities, and then the IdP and RPs could always link a user's (pseudo-)identities and accounts (i.e., both IdP untraceability and RP unlinkability are broken). That is, regardless of which $\mathcal{F}_{Acct*}()$ is adopted to determine $Acct_{i,j} = \mathcal{F}_{Acct*}(ID_{U_i}, ID_{RP_j})$, the IdP could always collude with RPs to track a user's login activities by analyzing the relationship between the user's identity and accounts derived at the RPs, unless $Acct_{i,j} = \mathcal{F}_{Acct*}(s_i, ID_{U_i}, ID_{RP_j})$ is adopted and s_i is a long-term user secret unknown to the colluding IdP and RPs.

As mentioned in Section II-B, privacy-preserving identity federation [15], [16], [34], [14], [18] prefers stronger privacy than user convenience, where a user secret is introduced to mask the relationship among the (pseudo-)identities and accounts. In this case, a user needs to install a browser extension to maintain the secret. Moreover, if the user accesses the services from different devices, additional operations are required to synchronize the secret among these devices.

Alternatively, MISO [12] introduces an extra fully-trusted mixer server other than the IdP to keep s_i , to calculate user pseudo-identities (or accounts) in each login. Then, in MISO

the mixer server can track all users' login activities, either colluding with RPs or not.

Support for the authorization code flow. In the authorization code flow of OIDC [1], the IdP does not directly return identity tokens. Instead, it generates an authorization code, which is forwarded to the target RP. The RP uses this code to retrieve identity tokens from the IdP.

$\mathcal{F}_{Acct*}()$, $\mathcal{F}_{PID_U}()$, $\mathcal{F}_{PID_{RP}}()$ and $\mathcal{F}_{Acct}()$, can be also integrated into the authorization code flow of OIDC to transform (pseudo-)identities in signed identity tokens. Then, in the flow an authorization code (but not the token) will be sent to the user's script, and to the RP finally. This code serves as an index to retrieve tokens by the RP, not disclosing any information about the user. On receiving an authorization code, the RP uses it as well as a credential issued by the IdP during the initial registration [1], to retrieve identity tokens from the IdP's token endpoint [2].

Meanwhile, to hide RP identities from the IdP in the retrieval of identity tokens in the authorization code flow, privacy-preserving credentials (e.g., ring signatures [59] or privacy passes [37], [38]) and anonymous communications (e.g., Tor [60] or oblivious proxies [61]) need to be adopted for RPs; Otherwise, IdP untraceability is broken.

Support for redirect UX. With redirect UX, an RP and the IdP exchange messages via HTTP redirection within one browser window. If we implement the UPPRESSO protocol with redirect UX, the following designs are needed to hide the visited RP's identity from the IdP during these HTTP redirections. These designs work either in the implicit flow

of OIDC or the authorization code flow.

In an OIDC system without the protections of user privacy, the RP's identity and redirection endpoint URL (i.e., ID_{RP} and $ReEnptUrl_{RP}$) are sent as query parameters in the HTTP redirection to the IdP, to request identity tokens. On the contrary, in UPPRESSO when a user clicks and redirects the browser from the RP-login page to the IdP server, ID_{RP} and $ReEnptUrl_{RP}$ are not sent. Meanwhile, $Cert_{RP}$ and the scope of requested user attributes are carried as URL fragments, so they are processed locally within the browser and not sent to the IdP web server. Besides, the RP suppresses the HTTP referer header (e.g., by setting `rel="noreferrer"`) of this redirection, which prevents the IdP server from receiving the RP's domain in the HTTP request of this redirection.

In the redirected page the user-i script verifies $Cert_{RP}$, extracts ID_{RP} and $ReEnptUrl_{RP}$, calculates PID_{RP} , and then requests an identity token from the IdP's authorization endpoint. Then, to forward the signed token to the RP, in the original OIDC protocol the IdP server constructs a server-side HTTP redirection to $ReEnptUrl_{RP}$. However, in UPPRESSO the IdP does not know the visited RP, so it returns the signed token to the user-i script but with an empty $ReEnptUrl_{RP}$. Then, the user-i script loads $ReEnptUrl_{RP}$ and t locally from `sessionStorage`, and submits the identity token and also t to the RP's redirection endpoint by a client-side redirection (e.g., by `window.location.href`).

Alternatives to generate ID_{RP} and bind $ReEnptUrl_{RP}$. In UPPRESSO the IdP generates random ID_{RP} and signs an RP certificate to bind ID_{RP} and $ReEnptUrl_{RP}$, which is verified by the user-i script and then ensures the relationship between the RP designated by an identity token and the URL to receive this token. It also guarantees that the target RP has already registered itself at the IdP and prevents unregistered RPs without such certificates from utilizing the IdP's services.

An alternative method to bind ID_{RP} and $ReEnptUrl_{RP}$ is a deterministic scheme to calculate unique ID_{RP} based on the RP's unambiguous name such as its domain. This can be achieved by encoding the domain with a hashing-to-elliptic-curves function [62], which provides collision resistance but not revealing the elliptic-curve discrete logarithm of the output. It generates a point on $\mathbb{E}$ as ID_{RP} , ensuring the *uniqueness* of $ID_{RP} = [r]G$ while keeping r unknown. In this case, in Step 1.2 the user-r script sends only the RP's redirection endpoint URL but not its RP certificate, and the user-i script calculates ID_{RP} by itself based on the RP's domain. Then, any RP can be visited through the SSO services, either registered or not.

However, if some RP changes its domain, for example, from `https://theRP.com` to `https://RP.com`, the accounts (i.e., $Acct = [ID_U]ID_{RP}$) will inevitably change. Thus, a user is required to perform special operations to migrate her account to the updated RP system. It is worth noting that the user operations cannot be eliminated in the migration to the updated one; otherwise, it implies two colluding RPs could link a user's accounts across these RPs.

Restriction of the targetOrigin mechanism. The user-i script forwards tokens to the user-r script, and the `postMessage` targetOrigin mechanism [23] restricts the recipient of the

forwarded identity tokens, to ensure that the tokens will be sent to the intended $ReEnptUrl_{RP}$, as specified in the RP certificate. The targetOrigin is specified as a combination of protocol, port (if not present, 80 for http and 443 for https), and domain (e.g., `RP.com`). The origin of the user-r script must accurately match the targetOrigin to receive tokens.

The targetOrigin mechanism does not check the whole URL path in $ReEnptUrl_{RP}$, but it introduces no additional risk. Consider two RPs in one domain but receiving tokens through different URLs, e.g., `https://RP.com/honest/tk` and `https://RP.com/malicious/tk`. This mechanism cannot distinguish them. Because a browser controls access to web resources following the same-origin policy (SOP) [63], a user's resources in the honest RP web server is accessible to the malicious server. For example, it could steal cookies by `window.open('https://RP.com/honest').document.cookie`, even if the honest RP restricts only the HTTP requests to specific paths are allowed to access its cookies. So this risk is caused by the SOP model of browsers but not our designs, and commonly exists in SSO services accessed from COTS browsers [2], [6], [53], [21], [22].

VIII. CONCLUSION

This paper presents UPPRESSO, an untraceable and unlinkable privacy-preserving SSO system for protecting a user's online profile across RPs against both a curious IdP and colluding RPs. We propose an identity-transformation approach and design algorithms satisfying the requirements of security and privacy: (a) $\mathcal{F}_{Acct*}()$ determines a user's account, unique at an RP and unlinkable across RPs, (b) $\mathcal{F}_{PID_{RP}}()$ protects a visited RP's identity from the curious IdP, (c) $\mathcal{F}_{PID_U}()$ prevents colluding RPs from linking a user's multiple logins visiting different RPs, and (d) $\mathcal{F}_{Acct}()$ derives a user's permanent account from ephemeral pseudo-identities.

The identity transformations are integrated into the widely-adopted OIDC protocol, while maintaining user convenience and security guarantees of SSO services. Our experimental evaluations of the UPPRESSO prototype demonstrate its efficiency. For example, when the scripts are not cached, in UPPRESSO a login takes 187 ms in average when the IdP, the visited RP, and user browsers are deployed in a virtual private cloud, or 372 ms when a user visits remotely. Compared with MITREid Connect, it introduces an overhead of 89 – 150 ms to prevent two privacy threats.

REFERENCES

- [1] N. Sakimura, J. Bradley, M. Jones, B. de Medeiros, and C. Mortimore, *OpenID Connect core 1.0 incorporating errata set 1*, The OpenID Foundation, 2014.
- [2] D. Hardt, *RFC 6749: The OAuth 2.0 authorization framework*, Internet Engineering Task Force, 2012.
- [3] J. Hughes, S. Cantor, J. Hodges, F. Hirsch, P. Mishra, R. Philpott, and E. Maler, *Profiles for the OASIS security assertion markup language (SAML) v2.0*, OASIS, 2005.
- [4] T. Hardjono and S. Cantor, *SAML V2.0 subject identifier attributes profile version 1.0*, OASIS, 2018.
- [5] P. Grassi, E. Nadeau, J. Richer, S. Squire, J. Fenton, N. Lefkowitz, J. Danker, Y.-Y. Choong, K. Greene, and M. Theofanos, *SP 800-63C: Digital identity guidelines: Federation and assertions*, National Institute of Standards and Technology (NIST), 2017.

- [6] D. Fett, R. Küsters, and G. Schmitz, "SPRESSO: A secure, privacy-respecting single sign-on system for the Web," in *22nd ACM Conference on Computer and Communications Security (CCS)*, 2015, pp. 1358–1369.
- [7] —, "Analyzing the BrowserID SSO system with primary identity providers using an expressive model of the Web," in *20th European Symposium on Research in Computer Security (ESORICS)*, 2015.
- [8] E. Maler and D. Reed, "The venn of identity: Options and issues in federated identity management," *IEEE Security & Privacy*, vol. 6, no. 2, pp. 16–23, 2008.
- [9] F. A. Services, "About Firefox Accounts," <https://mozilla.github.io/application-services/docs/accounts/welcome.html>, accessed August 20, 2019.
- [10] M. Kroschewski and A. Lehmann, "Save the implicit flow? Enabling privacy-preserving RP authentication in OpenID Connect," *Privacy Enhancing Technologies*, vol. 2023, no. 4, pp. 96–116, 2023.
- [11] S. Hammann, R. Sasse, and D. Basin, "Privacy-preserving OpenID Connect," in *15th ACM Asia Conference on Computer and Communications Security (AsiaCCS)*, 2020, pp. 277–289.
- [12] R. Xu, S. Yang, F. Zhang, and Z. Fang, "MISO: Legacy-compatible privacy-preserving single sign-on using trusted execution environments," in *8th IEEE European Symposium on Security and Privacy (EuroS&P)*, 2023.
- [13] C. Guo, F. Lang, Q. Wang, and J. Lin, "UP-SSO: Enhancing the user privacy of SSO by integrating PPID and SGX," in *International Conference on Advanced Computing and Endogenous Security (ICACES)*, 2021.
- [14] A. Dey and S. Weis, "PseudoID: Enhancing privacy for federated login," in *3rd Hot Topics in Privacy Enhancing Technologies (HotPETs)*, 2010.
- [15] Z. Zhang, M. Król, A. Sonnino, L. Zhang, and E. Rivière, "EL PASSO: Efficient and lightweight privacy-preserving single sign on," *Privacy Enhancing Technologies*, vol. 2021, no. 2, pp. 70–87, 2021.
- [16] M. Isaakidis, H. Halpin, and G. Danezis, "UnlimitID: Privacy-preserving federated identity management using algebraic MACs," in *15th ACM Workshop on Privacy in the Electronic Society (WPES)*, 2016, pp. 139–142.
- [17] G. Maganis, E. Shi, H. Chen, and D. Song, "Opaak: Using mobile phones to limit anonymous identities online," in *10th International Conference on Mobile Systems, Applications, and Services (MobiSys)*, 2012.
- [18] C. Paquin, *U-Prove technology overview v1.1*, Microsoft Corporation, 2013.
- [19] Hyperledger Fabric, "MSP implementation with Identity Mixer," <https://hyperledger-fabric.readthedocs.io/en/release-2.2/identmix.html>, accessed July 20, 2022.
- [20] W. Li and C. Mitchell, "Analysing the security of Google's implementation of OpenID Connect," in *13th International Conference on Detection of Intrusions and Malware & Vulnerability Assessment (DIMVA)*, 2016.
- [21] Google for Developers, "Google Identity: Integration considerations," <https://developers.google.com/identity/gsi/web/guides/integrate/>, accessed January 13, 2025.
- [22] Uber Developers, "OIDC Web SDK," <https://developer.uber.com/docs/consumer-identity/oidc/web>, accessed April 10, 2025.
- [23] The WHATWG Community, "HTML living standard: 9.3 cross-document messaging," <https://html.spec.whatwg.org/multipage/web-messaging.html>.
- [24] B. de Medeiros, M. Scurtescu, P. Tarjan, and M. Jones, *OAuth 2.0 multiple response type encoding practices*, The OpenID Foundation, 2014.
- [25] M. Asghar, M. Backes, and M. Simeonovski, "PRIMA: Privacy-preserving identity and access management at Internet-scale," in *52nd IEEE International Conference on Communications (ICC)*, 2018.
- [26] B. Diamond and J. Posen, "Succinct arguments over towers of binary fields," <https://eprint.iacr.org/2023/1784>, 2024.
- [27] J. Ernstberger, S. Chaliasos, G. Kadianakis, S. Steinhorst, P. Jovanovic, A. Gervais, B. Livshits, and M. Orru, "zk-Bench: A toolset for comparative evaluation and performance benchmarking of SNARKs," in *14th International Conference on Security and Cryptography for Networks (SCN)*, 2024.
- [28] W. Ma, Q. Xiong, X. Shi, X. Ma, H. Jin, H. Kuang, M. Gao, Y. Zhang, H. Shen, and W. Hu, "GZKP: A GPU accelerated zero-knowledge proof system," in *28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023, pp. 340–353.
- [29] J. Wang, G. Wang, and W. Susilo, "Anonymous single sign-on schemes transformed from group signatures," in *5th International Conference on Intelligent Networking and Collaborative Systems (INCoS)*, 2013.
- [30] J. Han, L. Chen, S. Schneider, H. Treharne, and S. Wesemeyer, "Anonymous single-sign-on for n designated services with traceability," in *23rd European Symposium on Research in Computer Security (ESORICS)*, 2018.
- [31] J. Han, L. Chen, S. Schneider, H. Treharne, S. Wesemeyer, and N. Wilson, "Anonymous single sign-on with proxy re-verification," *IEEE Transactions on Information Forensics and Security*, vol. 15, pp. 223–236, 2020.
- [32] D. Chaum, "Blind signatures for untraceable payments," in *CRYPTO*, 1982, pp. 199–203.
- [33] J. Camenisch and A. Lysyanskaya, "An efficient system for non-transferable anonymous credentials with optional anonymity revocation," in *EUROCRYPT*, 2001.
- [34] J. Camenisch and E. Herreweghen, "Design and implementation of the Idemix anonymous credential system," in *9th ACM Conference on Computer and Communications Security (CCS)*, 2002.
- [35] M. Chase, S. Meiklejohn, and G. Zaverucha, "Algebraic MACs and keyed-verification anonymous credentials," in *21st ACM Conference on Computer and Communications Security (CCS)*, 2014.
- [36] Z. Zhang, C. Xu, C. Jiang, and K. Chen, "TSAPP: Threshold single-sign-on authentication preserving privacy," *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 4, pp. 1515–1527, 2024.
- [37] A. Davidson, I. Goldberg, N. Sullivan, G. Tankersley, and F. Valsorda, "PrivacyPass: Bypassing Internet challenges anonymously," *Privacy Enhancing Technologies*, vol. 2018, no. 3, pp. 164–180, 2018.
- [38] Web Incubator CG, "TrustToken API," <https://github.com/WICG/trust-token-api>, accessed July 20, 2022.
- [39] M. Naor and O. Reingold, "Number-theoretic constructions of efficient pseudo-random functions," *Journal of the ACM*, vol. 51, no. 2, pp. 231–262, 2004.
- [40] S. Jarecki, A. Kiayias, and H. Krawczyk, "Round-optimal password-protected secret sharing and T-PAKE in the password-only model," in *AsiaCrypt*, 2014.
- [41] C. Guo, J. Lin, Q. Cai, W. Wang, F. Li, Q. Wang, J. Jing, and B. Zhao, "UPPRESSO: Untraceable and unlinkable privacy-preserving single sign-on services," <https://arxiv.org/abs/2110.10396>, 2022.
- [42] M. Freedman, Y. Ishai, B. Pinkas, and O. Reingold, "Keyword search and oblivious pseudorandom functions," in *2nd Theory of Cryptography Conference (TCC)*, 2005.
- [43] S. Jarecki, A. Kiayias, H. Krawczyk, and J. Xu, "Highly-efficient and composable password-protected secret sharing (or: How to protect your Bitcoin wallet online)," in *1st IEEE European Symposium on Security and Privacy (EuroS&P)*, 2016, pp. 276–291.
- [44] M. Freedman, Y. Ishai, B. Pinkas, and O. Reingold, "PESTO: Proactively secure distributed single sign-on, or how to trust a hacked server," in *5th IEEE European Symposium on Security and Privacy (EuroS&P)*, 2020.
- [45] S. Jarecki and X. Liu, "Efficient oblivious pseudorandom function with applications to adaptive OT and secure computation of set intersection," in *6th Theory of Cryptography Conference (TCC)*, 2009, pp. 577–594.
- [46] A. Bagherzandi, S. Jarecki, Y. Lu, and N. Saxena, "Password-protected secret sharing," in *18th ACM Conference on Computer and Communications Security (CCS)*, 2011, pp. 433–444.
- [47] D. Kales, C. Rechberger, T. Schneider, M. Senker, and C. Weinert, "Mobile private contact discovery at scale," in *28th USENIX Security Symposium*, 2019.
- [48] S. Jarecki, H. Krawczyk, and J. Resch, "Updatable oblivious key management for storage systems," in *26th ACM Conference on Computer and Communications Security (CCS)*, 2019, pp. 379–393.
- [49] J. Camenisch, A. de Caro, E. Ghosh, and A. Sorniotti, "Oblivious PRF on committed vector inputs and application to deduplication of encrypted data," in *23rd International Conference on Financial Cryptography and Data Security (FC)*, 2019, pp. 337–356.
- [50] J. Lin, B. Zhang, W. Wang, Q. Cai, J. Jing, and H. He, "Understanding the Identity-Transformation Approach in OIDC-Compatible Privacy-Preserving SSO Services," <https://arxiv.org/abs/2506.01325>, 2025.
- [51] S. Casacuberta, J. Hesse, and A. Lehmann, "SoK: Oblivious pseudorandom functions," in *7th IEEE European Symposium on Security and Privacy (EuroS&P)*, 2022.
- [52] D. Fett, R. Küsters, and G. Schmitz, "An expressive model for the web infrastructure: Definition and application to the BrowserID SSO system," in *35th IEEE Symposium on Security and Privacy (S&P)*, 2014.
- [53] J. Richer, "MITREid Connect v1.3.3," <http://mitreid-connect.github.io/index.html>, accessed August 20, 2021.
- [54] T. Berners-Lee, R. Fielding, and L. Masinter, *RFC 3986: Uniform resource identifier (URI): Generic syntax*, Internet Engineering Task Force, 2005.

- [55] The WHATWG Community, “HTML living standard: 12 web storage,” <https://html.spec.whatwg.org/multipage/webstorage.html>.
- [56] J. Eisinger and E. Stark, *W3C candidate recommendation: Referrer policy*, World Wide Web Consortium (W3C), 2017.
- [57] N. Sakimura, J. Bradley, and M. Jones, *OpenID Connect dynamic client registration 1.0 incorporating errata set 2*, The OpenID Foundation, 2023.
- [58] F. Indutny, “Fast elliptic curve cryptography in plain JavaScript,” <https://www.npmjs.com/package/elliptic>, accessed July 20, 2025.
- [59] A. Bender, J. Katz, and R. Morselli, “Ring signatures: Stronger definitions, and constructions without random oracles,” in *3rd Theory of Cryptography Conference (TCC)*, 2006, pp. 60–79.
- [60] R. Dingledine, N. Mathewson, and P. Syverson, “Tor: The second-generation onion router,” in *13th USENIX Security Symposium*, 2004, pp. 303–320.
- [61] M. Thomson and C. Wood, *RFC 9458: Oblivious HTTP*, Internet Engineering Task Force, 2024.
- [62] A. Faz-Hernandez, S. Scott, N. Sullivan, R. Wahby, and C. Wood, *draft-irtf-cfrg-hash-to-curve-16: Hashing to elliptic curves*, Internet Engineering Task Force, 2022.
- [63] W3C Web Security, “Same origin policy,” https://www.w3.org/Security/wiki/Same-Origin_Policy.